



University of Antwerp
| Faculty of Applied
Engineering

Lab 6 – Agents

Distributed Systems

Andreas Gavrielides

University of Antwerp – imec, IDLab – Faculty of Applied Engineering, Belgium

Overview of lab sessions

- **Lab 1: DONE**
 - Introductory lab & TCP/UDP
 - Setup Remote nodes
- **Lab 2: DONE**
 - REST protocol
 - Remote nodes
 - 2 sessions
- **Lab 3: Project: DONE**
 - Naming server
 - Remote nodes
 - 2 sessions
- **Lab 4: Project -> In progress**
 - Discovery service
 - Remote nodes
 - 2 sessions
- **Lab 5: Project -> In progress**
 - Replication
 - Remote nodes
 - 4 sessions
- **Lab 6: Project -> Today**
 - Agents
 - Remote nodes
 - 1 session
- **Lab 7: Project**
 - GUI
 - Remote nodes
 - 1 session
- **Finishing project**
 - Remote nodes
 - 4 sessions

Lab Agents [project]

- **Goal of the session:**

- To enable `synchronization` in system Y,
by creating two agents: Sync and Failure

- **Team work:**

- Work in your groups (available on Blackboard), **divide the work!**

- **Phases:**

1. Sync agent
2. Failure agent

Agents

- The last phase of the project
- It enables overall synchronization among nodes in System Y
- To enable synchronization -> two agents, i.e., the Sync Agent and the Failure Agent:
 - Sync agents has to make sure that all available files in the network are at the right place, checking every node and all the files this node owns.
 - Failure agent is started as soon as a node failure is detected. The responsibility of this agent is to transfer all the files from a failed node to the new owner, and to update the whole file list.

Sync Agent

- Use Java Agent Framework – software framework for development of intelligent agents in Java
- Each node in System Y has its own Sync Agent
- Agents duties:
 1. To create a list of all files that the node owns
 2. To update the list of files based on the state of **the next node**, i.e., this list should be synchronized **with the list stored on the next node**
 - (Thus the sync agent will ask periodically its next neighbouring node its sync agent for its file list).
 3. To lock a file, so it can be safely downloaded
 - **Distributed File Locking** – this refers to the concept of having multiple copies of a file on several nodes, and as soon as one file is opened for writing, all other copies are locked. This prevents a file from being modified on multiple nodes at the same time by several users.

Failure Agent

1. **Use Java Agent Framework – software framework for development of intelligent agents in Java)**
2. **The Failure Agent should be passed from one node in system Y to another**
3. **Agent's duties:**
 1. To transfer the ownership of the files owned by the failing node, to the new owner
 2. On each node, the Failure Agent should check if there are files that are owned by the failing node, and if so, the file(s) should be transferred to the new owner, and the log file should be updated on that change

Targets

■ Target 1:

- Every node should have a list (i.e., data structure) of all files in the system Y
- This list is synchronized by **Sync Agent**
- This list should consist of:
 1. Filename
 2. Information on whether there is an active lock on the node or not

■ Target 2:

- To develop Sync Agent class that implements the Runnable and Serializable interface:
 - This Sync Agent stores a data structure (Agent's list), i.e., a list of all files in the network
 - It performs *run()* method by:
 - Listing all files owned by the node at which this agent runs,
 - If one of the owned files is not added to the list, the list needs to be updated
 - Update the list stored by the node based on the agent's list
 - If there is a lock request on the current node, and the file is not locked on the agent's list, locking should be enabled on the node and the list should be synchronized accordingly
 - Remove the lock when it is not needed anymore, and update local file list accordingly

Targets

■ Target 3:

- To develop Failure Agent class that implements the Runnable and Serializable interface:
 - The failing node id is added to the constructor,
 - The current node id is added to the constructor (the node that started Failure agent)
 - The *run()* method will perform:
 1. Reading the file list of the current node
 2. If the failing node is the owner of the file, then the file should be transferred to the new owner, but there are two options:
 1. **Option 1:** if the new owner doesn't have a copy of this file already, then the file transfer can be done without any problems. The logs should be updated with a new download location.
 2. **Option 2:** if the file is already stored on the new owner, only the log should be updated
- Terminate the Failure Agent when it passed all nodes in the ring topology
 - If the node id is equal to the node id that started the agent -> terminate the Agent

Specifications

1. Every node creates a REST object that implements a method that:

1. Receives an agent as a parameter
2. Creates a thread from a received agent
3. Starts the thread
4. Waits until the thread is finished
5. Executes the REST method on the next node (unless the agent needs to be terminated)

2. During the system launch phase:

1. Sync Agent should be created
2. Sync agent will run infinitely

3. When Failure is detected:

1. Failure Agent should be created
2. Method described in (1) should be executed, and failure agent should be passed as a parameter.